Open Mercato on Windows

One-command development environment — Setup & Troubleshooting Manual

1. What this is

A single double-click of `scripts\windows\start-windows.bat` turns a Windows machine into a complete Open Mercato development environment running in Docker: the **application** (port 3000, hot reload), the **MCP server** (port 3001), the **OpenCode AI agent** (port 4096), plus PostgreSQL, Redis, and Meilisearch. The database is created and seeded automatically. The AI assistant (`Ctrl`+`K` in the app) works out of the box once you provide an LLM provider API key during setup.

Every step is idempotent: re-running `start-windows.bat` is always safe, skips what is already done, and repairs what is missing.

There are **three launchers**, all sharing the same engine — pick by container runtime:

Launcher	Use when
`start-windows.bat`	Default: auto-detects Docker Desktop or Rancher Desktop; installs Docker Desktop on a clean machine
`start-windows-rancher.bat`	Your organization uses **Rancher Desktop** (typical where Docker Desktop licensing is not permitted). With WSL2 already present it installs Rancher per-user — **no administrator rights at all**
`start-windows-docker.bat`	Force **Docker Desktop** explicitly

Unsure whether a locked-down machine will let setup succeed? Run `scripts\windows\check-windows.bat` first — a read-only preflight that audits policies, WSL2, network egress (proxy/TLS interception, download hosts), antivirus, and ports, and ends with a READY / CAVEATS / LIKELY-BLOCKED verdict. It changes nothing.

2. Requirements

Requirement	Details
Windows	Windows 10 2004+ (build 19041, incl. 22H2/Enterprise) or Windows 11
CPU virtualization	Intel VT-x / AMD-V enabled in BIOS/UEFI (usually already on)
Memory / disk	**16 GB RAM recommended, 12 GB minimum** (the launcher checks and refuses below 12 GB — override with `-SkipResourceCheck`); ~20 GB free disk for the first run, 40 GB+ comfortable
Container runtime	Docker Desktop *or* Rancher Desktop — installed automatically if missing

Administrator rights	**Only** for the one-time install of Docker/WSL2. Not needed afterwards, and not needed at all when IT pre-installed the runtime (see `-NoAdmin`)
LLM API key	One of: OpenAI, Anthropic, Google, Azure OpenAI / AI Foundry, OpenRouter, DeepInfra, Groq, Together, Fireworks, LiteLLM, Ollama, LM Studio

> **Not needed:** Node.js, winget / App Installer, Visual Studio Build Tools, WSL knowledge. Everything runs inside containers; installers are downloaded directly from official sources when winget is unavailable.

3. Quick start

A. You have the repository (git clone or ZIP)

1. Double-click `scripts\windows\start-windows.bat` in the repository.
2. If Docker/WSL2 must be installed, approve the UAC (administrator) prompt. If Windows asks to restart — restart; **setup resumes automatically after you log back in**.
3. When asked, pick your LLM provider and paste its API key (input is hidden). This is required — it powers the AI assistant.
4. Wait. The **first boot takes about 10 minutes** (up to ~20 on slow disks) while dependencies install and the database is seeded. Every later start takes seconds.
5. At the end a green summary prints all URLs and the **superadmin login and password**. Open `http://localhost:3000/backend` and sign in.

B. Completely clean machine (no Git, no repo)

Download just `start-windows.bat` (from the repository's `scripts/windows/` folder on GitHub) and double-click it. It fetches the launcher, installs Git (or falls back to a ZIP download of the repository), clones into `%USERPROFILE%\open-mercato`, and continues as above.

4. What happens, step by step

Step	Expected duration
Preflight (Windows version, virtualization, AppLocker check)	instant
Prerequisites report + install of anything missing (Git, WSL2, Docker/Rancher)	checks instant; installs 5–15 min
Reboot check (auto-resume after login if needed)	instant
Container engine start	30–90 s; first-ever launch up to 5 min
Repository download (standalone mode only)	1–3 min
`.env` generation (secrets created once, never overwritten)	instant
LLM provider prompt	waits for you

`docker compose up` (first run builds images)	seconds; first run 10+ min
Database health, application readiness, AI services	first boot ~10 min total; later runs seconds

During long waits the console shows a spinner with elapsed time and a plain-language status. Build progress is also live at `http://localhost:4000` (blank until the dependency install finishes — that is expected).

> **Memory (WSL2):** both Docker Desktop and Rancher Desktop run every container inside one WSL2 virtual machine, and Windows sizes that VM badly for this stack (Windows 11 caps it at 8 GB — too small; Windows 10 lets it take 80% of RAM — starves the host). On machines under 20 GB RAM the launcher therefore writes a cap into `C:\Users\<you>\.wslconfig` before the engine first starts: `memory=` (total RAM minus 6 GB, at least 8 GB — 10 GB on a 16 GB machine), `swap=8GB`, `autoMemoryReclaim=gradual`. It never touches an existing `memory=` line; edit or delete those lines to tune or revert (then quit the runtime and run `wsl --shutdown` to apply).

5. After setup — URLs and daily commands

Service	URL
Admin app	`http://localhost:3000/backend`
Build/status splash	`http://localhost:4000`
MCP server health	`http://localhost:3001/health`
OpenCode agent health	`http://localhost:4096/global/health`
Keycloak (dev SSO)	`http://localhost:8080` (admin/admin)

Action	Command
Start / restart the stack	`scripts\windows\start-windows.bat` (or `start-windows-rancher.bat` / `start-windows-docker.bat` to pin the runtime)
Read-only machine audit (before or after problems)	`scripts\windows\check-windows.bat`
Stop (data preserved)	`scripts\windows\stop-windows.bat`
Status of all services	`powershell scripts\windows\start-dev.ps1 -Status`
Follow application logs	`powershell scripts\windows\start-dev.ps1 -Logs`
Rebuild the Docker image (after Dockerfile changes)	`powershell scripts\windows\start-dev.ps1 -Rebuild`
Full reset — **DELETES all data**	`powershell scripts\windows\start-dev.ps1 -Reset`
Run without administrator rights	`powershell scripts\windows\start-dev.ps1 -NoAdmin`

Prefer/install Rancher Desktop instead of Docker Desktop	`powershell scripts\windows\start-dev.ps1 -Runtime rancher`
Test run with zero changes	`powershell scripts\windows\start-dev.ps1 -DryRun`

6. Working in your IDE

Edit the project folder (e.g. `C:\Users\<you>\open-mercato`) with any Windows IDE — VS Code, JetBrains, Cursor. Saving a file hot-reloads the app through Docker's bind mount. **WSL never enters your workflow**: `/mnt/c/...` is only how a WSL shell would see your disk — do not run the project from a WSL terminal, and do not keep a second copy inside a WSL distro.

> **IntelliSense caveat:** `node_modules` lives in a container volume, not in your Windows folder — so TypeScript autocomplete, go-to-definition into dependencies, and ESLint need one of the setups below.

Option	How	When
Attach VS Code to the container (recommended)	Install the "Dev Containers" extension → *Dev Containers: Attach to Running Container…* → pick the `app` container → open `/app`	Full IntelliSense with zero host tooling — the right default on corporate machines
Host `node_modules` for the editor only	Install Node 24, run `corepack enable` and `yarn install` once in the project folder	You prefer a plain host IDE; the app still runs in containers (warnings about optional native modules are harmless)
Full WSL2-native development	Clone inside an Ubuntu WSL distro, IDE via Remote-WSL (see the WSL2 guide in the docs)	Power users wanting maximum filesystem performance; requires a WSL distro and admin

Tip: "Go to definition" on `@open-mercato/*` imports lands in the monorepo package *sources* — that is expected and usually what you want. After editing a *package* (not the app), also restart the MCP container: `docker compose -f docker-compose.fullapp.dev.yml restart mcp`.

7. Troubleshooting

> **First rule:** the setup transcript is at `%TEMP%\open-mercato-setup\start-dev-*.log`, and container logs are one command away: `docker compose -f docker-compose.fullapp.dev.yml logs -f app` (replace `app` with `mcp`, `opencode`, `postgres` …).

Installation problems

Symptom	Cause & fix
"App package is not supported…" when installing winget manually	Do **not** install winget — it is not needed. The launcher downloads official installers directly. Just run `start-windows.bat` again.

Downloads fail (corporate proxy / air-gapped network)	Either set `HTTPS_PROXY` for the session, or place the official installers (Git for Windows, Docker Desktop or Rancher Desktop, `wsl_update_x64.msi`) in an `installers` folder in the repository root (or point `OM_INSTALLERS_DIR` at them). The launcher validates and uses them with no network access.
"CPU virtualization appears disabled"	Enable Intel VT-x / AMD-V (SVM) in BIOS/UEFI. On managed machines, ask IT.
"PowerShell is running in ConstrainedLanguage mode"	An AppLocker/WDAC policy restricts PowerShell. Ask IT to allow this script or run it from an exempted path.
No administrator rights at all	Ask IT to install *either* Docker Desktop (WSL2 backend; add your account to the `docker-users` group) *or* Rancher Desktop (dockerd/moby engine). Then run `start-dev.ps1 -NoAdmin` — everything else works unprivileged (the repo is fetched as ZIP if Git is missing).
UAC prompt cancelled by mistake	Just run `start-windows.bat` again.

Engine and container problems

Symptom	Cause & fix
"Docker engine did not become ready within 5 minutes"	Open Docker Desktop from the Start menu and finish first-run dialogs (sign-in can be skipped). If it says *"WSL 2 installation is incomplete"*: run `wsl --update` in an elevated prompt or install `https://aka.ms/wsl2kernel`, then re-run.
Both Docker Desktop and Rancher Desktop installed	The launcher uses whichever is already running, else Docker Desktop, and keeps the docker CLI context aligned. Force a choice with `-Runtime docker` or `-Runtime rancher`.
Rancher Desktop: `docker compose` not working	Rancher must run the **dockerd (moby)** engine (the launcher requests this itself via `rdctl`). In the Rancher UI: Preferences → Container Engine → dockerd (moby).
"WARNING: DOCKER_INSECURE_NO_IPTABLES_RAW is set"	Benign Docker Desktop networking notice — safe to ignore. It does not stop setup.
"The '<service>' container is crash-looping"	The launcher prints the failing container's last log lines — the real error is there. Most common: `.env` credentials no longer match an existing database volume → run `-Reset` (deletes data) or restore the old `.env`. Stop the restart churn with `stop-windows.bat`, fix, re-run.
"Port 3000/3001/4096… already in use"	Another process owns the port (the launcher names it). Close it, or override ports in the repo-root `.env`: `APP_PORT`, `MCP_PORT`, `OPENCODE_PORT`, `OM_DEV_SPLASH_PORT`, `KEYCLOAK_PORT` — the launcher follows these automatically.

Containers keep dying / "app container crash-looping" on a 16 GB machine	Usually the WSL2 VM running out of memory. Check that `C:\Users\<you>\.wslconfig` has the launcher's `[wsl2]` memory cap (see the note in section 4), close other heavy applications during the first build, and if you raised or removed the cap yourself, restore it — then quit the runtime, `wsl --shutdown`, re-run the launcher.
Image build fails downloading (apk / yarn / TLS errors) on a corporate network	The launcher diagnoses this itself: it names the cause (VM DNS breakage, TLS-intercepting proxy, blocked host), auto-exports the proxy's root CA into `docker\certs\` and retries. If it cannot: see `docker\certs\README.md` for the manual certificate steps, ask IT to allow `dl-cdn.alpinelinux.org`, or set `ALPINE_MIRROR` in `.env` to an internal mirror.

Application and AI problems

Symptom	Cause & fix
First boot "looks stuck"	It almost never is: the first boot installs dependencies, builds ~20 packages, and seeds the database inside the container (~10 min, up to ~20). Watch `http://localhost:4000` or the app logs. The launcher fails loudly if something actually breaks.
Browser cannot reach `http://localhost:3000`	Confirm the app container is running/healthy: `-Status`. If a firewall/VPN client filters localhost, allow local connections to ports 3000/4000.
AI chat (Ctrl+K) answers with authorization errors / MCP "disconnected"	The MCP key rotates when the database is reset — the OpenCode container reads it only at start. Run: `docker compose -f docker-compose.fullapp.dev.yml restart opencode`. Verify with `curl http://localhost:4096/mcp` → should show `"status":"connected"`.
AI chat gives provider/model errors	Check your provider key in the repo-root `.env` (e.g. `OPENAI_API_KEY`) and `OM_AI_PROVIDER`. After changing them: `docker compose -f docker-compose.fullapp.dev.yml restart opencode app`.
Login credentials from the summary do not work	If you recreated `.env` while keeping the old database volume, the printed password may differ from the seeded one (default: `password`). Cleanest fix: `-Reset` then a fresh run.
Restarted only the `app` container and AI tools act strangely	The app rebuild rewrites shared volumes under the MCP server. Restart it too: `docker compose -f docker-compose.fullapp.dev.yml restart mcp`.

8. FAQ

How long does it take?

First run on a clean machine: ~15–35 minutes total (installers + possible reboot + ~10 min first boot). Every later start: under a minute plus engine startup.

Do I need administrator rights?

Only once, to install Docker/WSL2. If IT pre-installed them, never — use `-NoAdmin`.

We are not allowed to use Docker Desktop (licensing). What now?

Use Rancher Desktop — fully supported. Double-click `start-windows-rancher.bat`: it installs Rancher (per-user, without admin, when WSL2 is already present) or uses an existing one. It must use the dockerd (moby) engine — the launcher requests this itself.

Where are my login credentials?

In the green summary at the end of setup, and in the repo-root `.env` (`OM_INIT_SUPERADMIN_EMAIL` / `OM_INIT_SUPERADMIN_PASSWORD`). They are not written to any log file.

Where does my data live? How do I wipe it?

In named Docker volumes (they survive stops and reboots). Wipe everything with `powershell scripts\windows\start-dev.ps1 -Reset` (asks for typed confirmation).

How do I update to the latest code?

With Git: `git pull`, then `start-windows.bat`. Installed from ZIP: download a fresh ZIP and run the launcher inside it (data lives in Docker volumes, not in the folder).

Can I change the AI provider later?

Yes — edit the repo-root `.env` (provider key + `OM_AI_PROVIDER`, optionally `OM_AI_MODEL`), then restart: `docker compose -f docker-compose.fullapp.dev.yml restart opencode app`.

Which ports are used?

3000 (app), 4000 (build splash), 3001 (MCP), 4096 (OpenCode), 8080 (Keycloak). All overridable in `.env`.

Does it work offline?

Installers can be pre-seeded (the `installers` folder), but the first boot needs the internet for Docker images and npm packages. After the first successful boot, day-to-day work is local.

How do I remove everything?

`-Reset` (removes containers and data volumes), delete the repository folder, and uninstall Docker/Rancher Desktop from Windows Settings if no longer needed.
